

DEVELOPMENT OF REINFORCEMENT LEARNING CONTROL OF A BRUSHLESS DC MOTOR ACTUATED PARALLELOGRAM ARM REHABILITATION ROBOT

BY

YUNUS Mujahid Olalekan
(EEG/2018/096)



A THESIS SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE AWARD OF BACHELOR OF SCIENCE IN
ELECTRONIC AND ELECTRICAL ENGINEERING

FACULTY OF TECHNOLOGY
OBAFEMI AWOLOWO UNIVERSITY
ILE-IFE, OSUN STATE, NIGERIA

September 2025

AUTHORIZATION TO COPY

OBAFEMI AWOLOWO UNIVERSITY, ILE-IFE HEZEKIAH OLUWASANMI LIBRARY UNDERGRADUATE THESIS

AUTHOR: YUNUS Mujahid Olalekan

TITLE: Development of Reinforcement Learning Control of a Brushless DC Motor
Actuated Parallelogram Arm Rehabilitation Robot

DEGREE: B.Sc. (Electronic and Electrical Engineering)

YEAR: 2025

I, YUNUS Mujahid Olalekan, hereby authorize the Hezekiah Oluwasanmi Library to copy my thesis, in whole or part, in response to request from individual researchers and Organizations for the purpose of private studies or research.

Signature: _____

Date: _____

CERTIFICATION

I certify that this work was carried out by YUNUS Mujahid Olalekan in partial fulfilment of the requirements for the award of the degree of Bachelor of Science (B.Sc.), in the Department of Electronic and Electrical Engineering, Obafemi Awolowo University, Ile-Ife, Nigeria.

Prof. Kayode P. Ayodele

ACKNOWLEDGEMENTS

I acknowledge the support of my parents, uncles and aunts, brothers and sisters, friends and colleagues. I also acknowledge the guidance and mentorship of my lecturers especially my Supervisor, Prof K.P. Ayodele.

DEDICATION

I dedicate this final year project to my parents.

TABLE OF CONTENTS

LIST OF FIGURES	vii
LIST OF TABLES	viii
LIST OF ABBREVIATIONS	x
LIST OF ABBREVIATIONS	ix
ABSTRACT	xi
CHAPTER 1: INTRODUCTION	1
1.1 Background to the Study	1
1.2 Problem Statement	2
1.3 Aim and Objectives	3
1.4 Scope of the Project	3
CHAPTER 2: LITERATURE REVIEW	5
2.1 Introduction	5
2.2 Rehabilitation Robotics: An Overview	5
2.2.1 Stroke Rehabilitation and the Role of Robotics	5
2.2.2 Categories of Rehabilitation Robots	6
2.2.3 The PULSR Platform for Upper Limb Stroke Rehabilitation	6
2.3 Reinforcement Learning: A Theoretical Foundation	7
2.3.1 Classical Reinforcement Learning Algorithms	8
2.3.2 Deep Reinforcement Learning	8
2.4 State-of-the-Art in Reinforcement Learning for Rehabilitation Robotics	9
2.5 Review of Relevant Literature	10

CHAPTER 3: METHODOLOGY	13
3.1 Introduction	13
3.2 Description of the PULSR Robot	13
3.3 Kinematics and Dynamics of PULSR	14
3.4 Reinforcement Learning Problem Formulation	20
3.4.1 Environment	20
3.4.2 State (Observation Space)	20
3.4.3 Agent	21
3.4.4 Action Space	21
3.4.5 Reward Function	21
3.5 Development of the RL Environment	22
3.5.1 Data Collection for PULSR Environment Specifications	22
3.5.2 Data Processing Pipeline	24
3.5.3 Environment Specifications and Assumptions	25
3.5.4 The Standard Gymnasium Environment	27
CHAPTER 4: RESULTS AND DISCUSSION	30
4.1 RL Model Training	30
4.2 Testing the RL Model	32
CHAPTER 5: CONCLUSION	36
5.1 Summary of Work	36
5.2 Key Contributions	37
5.3 Challenges Encountered	37
5.4 Recommendations for Future Work	38
5.5 Conclusion	38

LIST OF FIGURES

3.1	Top View of the Platform for Upper Limb Stroke Rehabilitation (PULSR)	
	Robot	15
3.2	Geometry of PULSR	16
3.3	Kinematics of PULSR	16
3.4	Header of the Episode Data	23
3.5	Physical Outlook of the Environment	26
3.6	Environment Test with Gymnasium Standards	28
3.7	Environment Test with Actions	29
4.1	Excellent Test Result 1	33
4.2	Excellent Test Result 2	33
4.3	Excellent Test Result 3	34
4.4	Woeful Test Result 1	34
4.5	Woeful Test Result 2	35
4.6	Woeful Test Result 3	35

LIST OF TABLES

3.1	Action Mapping	21
3.2	Reward Function based on Distance	22
3.3	Environment Methods and Their Purpose	27

LIST OF ABBREVIATIONS

LIST OF ABBREVIATIONS

PULSR	Platform for Upper Limb Stroke Rehabilitation
RL	Reinforcement Learning
DQN	Deep Q-Network
DOF	Degree of Freedom
LSTM	Long Short-Term Memory
EMG	Electromyography
SARSA	State-Action-Reward-State-Action
TD	Temporal Difference
PPO	Proximal Policy Optimization
BLDC	Brushless Direct Current
CSV	Comma-Separated Values

ABSTRACT

Stroke rehabilitation requires intensive, repetitive, and adaptive training to restore upper limb function. Conventional rehabilitation robots provide consistent therapy but are limited by rigid, pre-programmed controllers that cannot adapt to the unique recovery patterns of individual patients. This project addresses this limitation by developing a reinforcement learning (RL) control strategy for the Platform for Upper Limb Stroke Rehabilitation (PULSR), a brushless DC motor-actuated parallelogram arm robot.

The study began with an extensive review of state-of-the-art applications of RL in rehabilitation robotics and an analysis of the kinematics and dynamics of the PULSR platform. Due to limitations in the force sensors, the environment was modeled using the robot's kinematics. A reinforcement learning problem was formulated with a defined state space, action space, and reward function. A custom RL environment was then developed in compliance with Gymnasium standards, enabling training and evaluation of RL algorithms.

A Deep Q-Network (DQN) was implemented as a benchmark model. Training over 100,000 timesteps yielded an average reward of 230.5 per episode, demonstrating the agent's ability to learn effective trajectory-tracking strategies within the simulated environment. These results highlight the feasibility of reinforcement learning as an adaptive control method for rehabilitation robots.

Although real-world deployment was constrained by sensor unreliability and interfacing challenges between the existing C control code and Python-based RL frameworks, the project establishes a strong foundation for future work. By integrating reinforcement learning into rehabilitation robotics, this study contributes to the development of intelligent, patient-specific controllers capable of enhancing recovery outcomes for stroke survivors.

Keywords: Rehabilitation Robotics, Reinforcement Learning, Brushless DC Motor, Parallelogram Arm, Deep Q-Network, Adaptive Control

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

Stroke remains one of the leading causes of long-term disability worldwide. Survivors often experience significant motor impairments, particularly in the upper limbs, which affect their independence and quality of life. Rehabilitation robots have emerged as a promising solution to provide consistent, repetitive, and intensive therapy, complementing the efforts of physiotherapists and improving patient recovery outcomes.

Among the various categories of rehabilitation robots, parallelogram arm robots have gained considerable attention because of their ability to mimic the natural biomechanics of the human arm. When actuated by high-performance motors, such robots can provide precise, repeatable motions necessary for effective rehabilitation.

The Platform for Upper Limb Stroke Rehabilitation (PULSR) is a brushless DC motor-actuated parallelogram arm robot designed to support stroke survivors in regaining motor function. While existing control strategies for such robots often rely on rigid pre-programmed trajectories or simple feedback controllers, these approaches fail to adapt to the unique recovery pace and input from individual patients.

Recent advancements in Reinforcement Learning (RL) — a branch of artificial intelligence that enables machines to learn optimal actions through interaction with their

environment — offer an opportunity to develop adaptive, patient-specific controllers. By integrating RL into PULSR’s control system, the robot can anticipate and augment user input in real-time, thereby personalizing therapy and promoting active patient engagement.

1.2 Problem Statement

Stroke survivors often experience severe motor impairments that limit their independence and quality of life. Rehabilitation robots, such as the Platform for Upper Limb Stroke Rehabilitation (PULSR), provide a means of delivering repetitive and intensive therapy to support recovery. However, the effectiveness of these robots is limited by their reliance on pre-programmed control strategies that do not adapt to individual patient progress.

Existing control approaches lack the ability to learn from patient interaction or provide personalized assistance, resulting in therapy that is less effective and less engaging. While reinforcement learning (RL) has shown promise in other areas of robotics for enabling adaptive, patient-specific control, its application to upper-limb rehabilitation robots remains underexplored.

Therefore, the key problem addressed in this study is the absence of an intelligent, adaptive control strategy for the PULSR robot that can anticipate user input, adjust in real-time to patient performance, and enhance rehabilitation outcomes through reinforcement learning.

1.3 Aim and Objectives

The primary aim of this project is to develop reinforcement learning models for the PULSR brushless DC motor-actuated parallelogram arm rehabilitation robot, enabling the system to anticipate and augment user input for enhanced rehabilitation outcomes.

The objectives of this project are to:

1. Establish the state-of-the-art in reinforcement learning for robotic rehabilitation in order to identify existing approaches and highlight research gaps.
2. Investigate foundational theories and understand the kinematics and dynamics of the PULSR2 robot.
3. Assess the current ability of the PULSR2 robot to anticipate and augment user input and identify areas where RL can provide significant improvements.
4. Design and develop reinforcement learning models capable of providing these improvements.
5. Deploy and evaluate the performance of the RL models on the PULSR2 robot.

1.4 Scope of the Project

This project focuses on the development and evaluation of reinforcement learning control strategies for the PULSR robot. Specifically:

- The project leverages the kinematics of the PULSR robot for environment modeling, given current limitations in force sensor accuracy that prevent reliable use of full dynamics.
- Reinforcement learning algorithms are developed and trained in a simulation environment before deployment on the physical robot.
- A Deep Q-Network (DQN) is used as a benchmark RL model due to its proven effectiveness in discrete action spaces.
- Deployment is limited by interfacing challenges between the C# control codebase and Python-based RL implementation; therefore, only prototype integration is achieved.

The project does not extend to full-scale clinical trials with patients but rather demonstrates technical feasibility and lays groundwork for future translational studies.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter presents a review of relevant literature underpinning the development of reinforcement learning control for rehabilitation robotics. The review begins with an overview of rehabilitation robotics, focusing on upper limb devices and the PULSR platform. This is followed by a discussion of reinforcement learning theory and its extensions into deep reinforcement learning. State-of-the-art applications of RL in rehabilitation robotics are then examined, with emphasis on key research contributions between 2015 and 2020. Finally, the chapter highlights challenges in applying RL to rehabilitation robots, identifies research gaps, and sets the stage for the design and implementation presented in subsequent chapters.

2.2 Rehabilitation Robotics: An Overview

2.2.1 Stroke Rehabilitation and the Role of Robotics

Stroke is one of the leading causes of adult disability worldwide. Survivors often suffer motor impairments that significantly reduce their independence. Conventional physiotherapy is resource-intensive, requiring highly repetitive exercises that may be

difficult for therapists to deliver consistently. Rehabilitation robots provide a solution by enabling repetitive, intensive, and quantifiable training. They also reduce therapist workload and allow therapy to be tailored to individual recovery trajectories.

2.2.2 Categories of Rehabilitation Robots

Rehabilitation robots are broadly divided into:

- **Exoskeleton-based systems:** These wearable devices align with the patient's limb joints, directly guiding motion. They provide high levels of control but are often complex to fit and calibrate.
- **End-effector and parallelogram arm robots:** These devices interact with the patient at a single contact point, usually the hand or wrist, and guide movement indirectly. They are mechanically simpler and more adaptable to different patients.
- **Hybrid systems:** Combining elements of exoskeletons and end-effectors for greater flexibility.

Parallelogram arm robots, such as PULSR, offer the advantage of mechanically replicating the natural motion of the upper limb while minimizing complexity.

2.2.3 The PULSR Platform for Upper Limb Stroke Rehabilitation

The Platform for Upper Limb Stroke Rehabilitation (PULSR) is a brushless DC motor-actuated parallelogram robot designed to aid post-stroke patients in regaining arm

mobility. The robot leverages a parallelogram geometry that enables smooth and controlled arm movements. PULSR incorporates sensors including encoders and load cells to track user input. However, its control system has historically relied on hard-coded algorithms, limiting its adaptability. The introduction of reinforcement learning offers a means to transform PULSR into an intelligent, patient-adaptive rehabilitation device.

2.3 Reinforcement Learning: A Theoretical Foundation

Reinforcement learning (RL) is a computational approach to sequential decision-making where an agent learns by interacting with an environment. The fundamental components include:

- **States (s):** Representing the environment's situation at a given time.
- **Actions (a):** Choices available to the agent.
- **Rewards (r):** Scalar feedback signals guiding the agent's learning.
- **Policy (π):** A mapping from states to actions.
- **Value Functions:** Estimating expected returns from states or state-action pairs.

Through trial and error, the RL agent aims to maximize cumulative reward.

2.3.1 Classical Reinforcement Learning Algorithms

Early RL algorithms relied on tabular methods:

- **Dynamic Programming:** Requires complete knowledge of environment dynamics; impractical for real-world robotics.
- **Monte Carlo Methods:** Learn from sampled episodes but suffer from high variance.
- **Temporal Difference (TD) Learning:** Combines bootstrapping and sampling. Examples include SARSA and Q-learning, which estimate action-value functions to guide policies.

2.3.2 Deep Reinforcement Learning

The integration of deep neural networks with RL gave rise to Deep Reinforcement Learning (DRL), enabling learning in high-dimensional spaces:

- **Deep Q-Networks (DQN):** Approximate Q-values with neural networks; stabilize learning using target networks and experience replay.
- **Actor-Critic Methods:** Simultaneously learn a policy (actor) and a value function (critic). Examples include Deep Deterministic Policy Gradient (DDPG) for continuous control and Proximal Policy Optimization (PPO) for stable updates.
- **Asynchronous Methods (A3C/A2C):** Improve sample efficiency by training multiple agents in parallel.

These methods form the basis of modern adaptive control strategies in robotics.

2.4 State-of-the-Art in Reinforcement Learning for Rehabilitation Robotics

The Deep Deterministic Policy Gradient (DDPG) approach has been investigated by Zhang Y, Li S, Nolan KJ, and Zanutto D. in their work on reinforcement learning assist-as-needed control for robot-assisted gait training [Zhang, Li, Nolan, and Zanutto \(2020\)](#). This model enables adaptive assist-as-needed control, dynamically adjusting assistance based on patient performance. It improves motor learning by encouraging active patient participation and demonstrates robustness in handling dynamic patient responses. However, its application has been largely limited to gait training, with little focus on upper-limb-specific rehabilitation tasks. Moreover, it lacks long-term clinical validation for upper limb recovery and requires further exploration of its generalization across diverse patient populations.

A Q-learning based RL approach was discussed by Abdullah et al. [Hayat, Chittawadigi, Udai, and Saha \(2013\)](#). This method provides effective solutions for inverse kinematics in high-degree-of-freedom (DOF) systems, simplifying trajectory planning for complex upper limb movements. It has shown promise in simulations, offering accurate joint angle predictions. Nevertheless, the research has so far been confined to simulation-based validation, without real-world clinical testing. Additionally, it requires extensive parameter tuning, which reduces its generalizability across tasks, and it does not address patient-specific adaptation or real-time feedback mechanisms.

Another line of work integrates Deep RL with Long Short-Term Memory (LSTM) networks, as demonstrated in the study [Shah et al. \(2025\)](#). This approach leverages LSTM networks to model temporal dependencies, enabling smoother motion planning. It generalizes well across high-DOF systems with minimal manual tuning and achieves accurate predictions of joint angles for varied end-effector positions. However, this research is limited to solving inverse kinematics problems and does not directly address patient interaction or motor recovery. It also lacks clinical trials that validate its efficacy in real-world rehabilitation settings, and further work is needed to integrate bio-signals such as electromyography (EMG) for enhanced control.

Finally, Gaussian Process RL has been explored by [Hong, Chun, Kim, and Park \(2019\)](#). This model supports safe rehabilitation by facilitating collision-free motion planning and adapts trajectories based on patient-specific needs. It also reduces computational complexity compared to deep RL models. Despite these advantages, the study primarily focused on gait training, offering limited insights into upper limb rehabilitation. Furthermore, being relatively dated, it does not incorporate modern sensor technologies, and its effectiveness for upper limb tasks and long-term patient outcomes remains unvalidated.

2.5 Review of Relevant Literature

Rehabilitation robotics has advanced considerably over recent years, with a growing emphasis on adaptive and personalized control systems. Traditional control strategies, such as those based on iterative learning control (ILC) and fuzzy algorithms, have been applied to enhance the performance of rehabilitation robots. For instance, a simulation

study using ILC for an upper limb rehabilitation robot demonstrated improvements in task performance while highlighting issues like transient error growth and excessive control inputs that compromise patient safety [Faremi et al. \(2022\)](#). Similarly, fuzzy algorithm-based approaches have been employed to address the inverse kinematics challenges in hybrid systems, offering a means to compensate for disturbances in dynamic environments [Wang et al. \(2018\)](#). In parallel, significant research has been devoted to improving the accuracy and reliability of robot control through advanced kinematic and dynamic analyses. Work on inverse kinematics techniques for anthropomorphic limbs and full-body performance animation provides a robust theoretical foundation for real-time motion reconstruction and controller design [Tolani, Goswami, and Badler \(2000\)](#), [Unzueta, Peinado, Boulic, and Suescun \(2008\)](#). These studies, along with comprehensive surveys on Jacobian-based methods and damped least squares techniques [Buss \(2004\)](#), underline the complexity of designing control algorithms that can efficiently handle the redundancy and nonlinearity inherent in articulated systems. The integration of low-cost sensor technologies has also received considerable attention. For example, a technique for end-effector force estimation using link-integrated load cells was developed to provide a cost-effective alternative to expensive multi-axis force/torque sensors [Ayodele et al. \(2023\)](#). This work is particularly relevant to the PULSR2 platform, where affordability and accurate force feedback are critical. Additionally, documentation on the PULSR2 therapy robot control application highlights practical challenges in interfacing with real hardware and ensuring robust error handling in clinical environments [?](#) . More recent studies have explored supervisory and adaptive control frameworks for multibody mechanisms, offering insights into how automatic configuration and behavior control can be achieved in complex systems [Liégeois \(1977\)](#). In parallel, research on hybrid kinematic

mechanisms [He, Gao, and Sun \(2018\)](#), [Kucuk and Gungor \(2016a\)](#) and redundancy resolution for serial-parallel manipulators [?](#) has further enriched the understanding of how to combine multiple control modalities for enhanced performance. Despite these advances, current systems predominantly rely on fixed or slowly adaptive control methods. There is a notable gap between these conventional approaches and the potential for a fully adaptive system that can anticipate user input in real time. Reinforcement learning (RL) offers a promising alternative by enabling continuous online adaptation through interaction with the environment. Unlike traditional methods such as ILC or fuzzy control, an RL-based controller can learn from sensor data (as demonstrated in [Ayodele et al. \(2023\)](#)) and adjust its policy to mitigate transient errors and reduce excessive control efforts. This adaptive capability is supported by the robust kinematic and dynamic frameworks found in classical robotics texts and studies [Kucuk and Gungor \(2016b\)](#), [Merat \(1987\)](#), and by supervisory control approaches that have been successfully implemented in other robotic applications [Liégeois \(1977\)](#). In summary, while significant research has established the foundations of kinematics, dynamic control, and sensor integration for rehabilitation robots, there remains a gap in developing systems that can proactively anticipate user input and this gap revolves round safety, adaptability to changes in environment and deployability to real-world rehab sessions.

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter presents the methodology adopted for the development of a reinforcement learning (RL) control system for the PULSR brushless DC motor-actuated parallelogram rehabilitation robot. It describes the system's structure, its mathematical modeling through kinematics and dynamics, the formulation of the RL problem, and the development of an RL environment for training. The choice of reinforcement learning algorithms, particularly the Deep Q-Network (DQN) benchmark model, is also discussed along with the experimental procedure used for training and evaluation.

3.2 Description of the PULSR Robot

The Platform for Upper Limb Stroke Rehabilitation (PULSR) is a parallelogram arm robot actuated by a brushless DC motor. Its geometry is defined by link lengths of 22, 43, 43, 65, and 42 mm. The system includes:

Actuators: Brushless DC motor providing smooth and precise motion.

Sensors: Load cell, encoder, and force sensor for feedback.

Control Application: A C hardcoded application for robot operation.

PULSR's parallelogram design allows natural human arm-like movements, making it suitable for upper limb rehabilitation.

3.3 Kinematics and Dynamics of PULSR

From the PULSR kinematics as shown in the image, the position, $A(x, y)$ of the end effector within the robot workspace can be determined as a function of the angles at the active links ϑ_1 and ϑ_2 as follows:

$$\mathbf{A} = \begin{bmatrix} l_2 \cos \vartheta_2 + l_3 \cos \vartheta_1 \\ l_2 \sin \vartheta_2 + l_3 \sin \vartheta_1 \end{bmatrix} \quad (1)$$

Dynamics of PULSR2

The generalized dynamic model for a manipulator with a vector of joint angles given by $\mathbf{q} = [\theta_1, \theta_2]^T$ is:

$$\mathbf{B}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{F}\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) = \boldsymbol{\tau} - \mathbf{J}^T(\mathbf{q})\mathbf{F}_e \quad (2)$$

where:

$\mathbf{B}$ = Inertia matrix of the manipulator, $\mathbf{C}$ = Coriolis terms, $\mathbf{F}$ = Frictional terms, $\mathbf{g}$ = Gravitational

$\boldsymbol{\tau}$ = Motor torque, $\mathbf{J}$ = Jacobian matrix, $\mathbf{F}_e$ = Force at end effector.



Figure 3.1: Top View of the Platform for Upper Limb Stroke Rehabilitation (PULSR) Robot

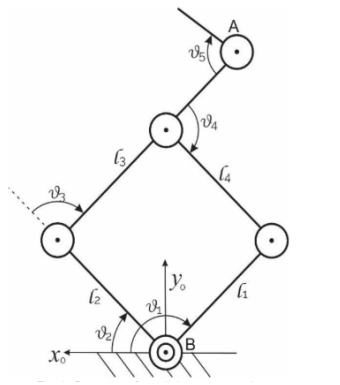


Figure 3.2: Geometry of PULSR

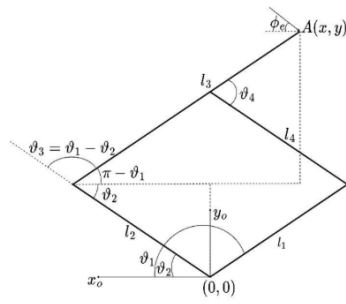


Figure 3.3: Kinematics of PULSR

Parallelogram arm rehabilitation robots are usually oriented to use a horizontal plane of motion. That orientation is assumed in this study, resulting in the elimination of the gravity component of Eq. 2. The impedance of the motor at rest is high enough that it requires significant effort by the user to set it into motion. Before the user has applied enough force to move the revolute joints, all links can be considered stationary. This implies a system at static equilibrium, for which $\dot{\mathbf{q}} = 0$, leading to a reduction of Eq. 2 to:

$$\mathbf{T} = \mathbf{J}^T \mathbf{F}_e \quad (3)$$

where $\mathbf{J}$, the Jacobian, is:

$$\mathbf{J} = \begin{bmatrix} \frac{\delta x}{\delta \theta_1} & \frac{\delta x}{\delta \theta_2} \\ \frac{\delta y}{\delta \theta_1} & \frac{\delta y}{\delta \theta_2} \end{bmatrix} = \begin{bmatrix} -l_3 \sin \theta_1 & -l_2 \sin \theta_2 \\ l_3 \cos \theta_1 & l_2 \cos \theta_2 \end{bmatrix} \quad (4)$$

The torque, $\mathbf{T}$ in Eq. 3, is the vector of torques induced at joint B (Fig. 1) due to the force, $\mathbf{F}_e$ applied at the end effector by the user. Due to the high impedance of the motors, the links do not initially rotate. This absence of rotation makes the applied force cause a bending moment on the load cell in the actuated links causing a slight deflection. The force readings $\mathbf{F}_L$ due to this deflection is as a result of the reactive torque at the active joints which opposes the induced torque. Thus, varying the applied force varies the induced torque which in turn varies the deflection and force readings from the load cell. Hence, a relationship can be inferred between forces measured by the load cells $\mathbf{F}_L$ and induced torque $\mathbf{T}$ as follows:

$$\mathbf{T} = \mathbf{k}^T \mathbf{F}_L \quad (5)$$

where: $\mathbf{k}$ is a 2×1 matrix of constants that can be determined experimentally. The individual torques can then be calculated to be:

$$\begin{bmatrix} T_1 \\ T_2 \end{bmatrix} = \begin{bmatrix} -l_3 \sin \theta_1 & l_3 \cos \theta_1 \\ -l_2 \sin \theta_2 & l_2 \cos \theta_2 \end{bmatrix} \begin{bmatrix} F_e \cos \phi_e \\ F_e \sin \phi_e \end{bmatrix} \quad (6)$$

or,

$$T_1 = -l_3 F_e \cos \phi_e \sin \theta_1 + l_3 F_e \sin \phi_e \cos \theta_1 \quad (7a)$$

$$T_2 = -l_2 F_e \cos \phi_e \sin \theta_2 + l_2 F_e \sin \phi_e \cos \theta_2 \quad (7b)$$

From Eq. 3:

$$\begin{bmatrix} F_{ex} \\ F_{ey} \end{bmatrix} = (\mathbf{J}^T)^{-1} \begin{bmatrix} T_1 \\ T_2 \end{bmatrix} \quad (8)$$

And from Eq. 4:

$$\det(\mathbf{J}^T) = -l_2 l_3 \sin \theta_1 \cos \theta_2 + l_2 l_3 \cos \theta_1 \sin \theta_2 = -l_2 l_3 \sin(\theta_1 - \theta_2) \quad (9)$$

$$(\mathbf{J}^T)^{-1} = -\frac{1}{l_2 l_3 \sin(\theta_1 - \theta_2)} \begin{bmatrix} l_2 \cos \theta_2 & l_2 \sin \theta_2 \\ -l_3 \cos \theta_1 & -l_3 \sin \theta_1 \end{bmatrix} \quad (10)$$

Therefore:

$$\begin{bmatrix} F_{ex} \\ F_{ey} \end{bmatrix} = -\frac{1}{l_2 l_3 \sin(\theta_1 - \theta_2)} \begin{bmatrix} l_2 \cos \theta_2 & l_2 \sin \theta_2 \\ -l_3 \cos \theta_1 & -l_3 \sin \theta_1 \end{bmatrix} \begin{bmatrix} T_1 \\ T_2 \end{bmatrix} \quad (11)$$

On substituting Eq. 5, this gives:

$$F_{ex} = \frac{-k_1 F_{L1} \cos \theta_2 - k_2 F_{L2} \sin \theta_2}{l_3 \sin(\theta_1 - \theta_2)} \quad (12a)$$

$$F_{ey} = \frac{k_1 F_{L1} \cos \theta_1 + k_2 F_{L2} \sin \theta_1}{l_2 \sin(\theta_1 - \theta_2)} \quad (12b)$$

Expressing this in polar form,

$$\mathbf{F}_e = \begin{bmatrix} \sqrt{\frac{l_2^2 (k_1 F_{L1})^2 \cos^2 \theta_2 + (k_2 F_{L2})^2 \sin^2 \theta_2 + k_1 k_2 F_{L1} F_{L2} (l_2^2 \sin \theta_2 \cos \theta_1 + l_3^2 \sin \theta_1 \cos \theta_2) + l_3^2 (k_2 F_{L2} \sin \theta_1 + k_1 F_{L1} \cos \theta_1)^2}{(l_2 l_3 \sin(\theta_1 - \theta_2))^2}} \\ \arctan \left(\frac{-l_3 k_1 F_{L1} \cos \theta_1 + l_2 k_2 F_{L2} \sin \theta_1}{l_2 k_1 F_{L1} \cos \theta_2 + l_2 k_2 F_{L2} \sin \theta_2} \right) \end{bmatrix} \quad (13)$$

3.4 Reinforcement Learning Problem Formulation

The control of the Platform for Upper Limb Rehabilitation Robot is formulated as a Reinforcement Learning (RL) problem. The formulation consists of the following components:

3.4.1 Environment

The environment represents the rehabilitation platform and the interaction between the parallelogram arm robot and the patient's upper limb. Its dynamics are governed by the robot's kinematics, actuator commands, and human-robot interaction forces. Because of the terminal challenges posed by the force sensors, the dynamics of the robot was not considered in the environment model. The environment therefore only models the kinematics of the robot.

3.4.2 State (Observation Space)

At each time step, the agent receives observations from the environment. The state vector is defined as:

$$\mathbf{s} = [ax, ay, tx, ty, dist]$$

where:

- ax, ay : Actual (measured) end-effector position in Cartesian space,
- tx, ty : Target position in Cartesian space (rehabilitation goal),

- *dist*: Distance between actual and target positions (tracking error).

3.4.3 Agent

The agent is modeled as a neural network policy, which takes the state $\mathbf{s}$ as input and outputs an action. The policy is parameterized as $\pi_{\theta}(a|s)$, where θ are the network parameters to be optimized.

3.4.4 Action Space

The action space corresponds to the different directions of motion possible for the robot. The robot's action space is defined as a discrete set of four possible movements in Cartesian coordinates, namely moving north, east, south, or west, as shown in Table 3.1.

:

Table 3.1: Action Mapping

Action	Mapping
0	North (Increase y)
1	East (Increase x)
2	South (Decrease y)
3	West (Decrease x)

3.4.5 Reward Function

The agent receives a scalar reward signal $r(d)$ from the environment. The reward is designed to encourage accurate trajectory tracking and smooth, safe motion. The reward function is defined according to the distance between the robot's end-effector and the

target point. As shown in Table 3.2, the agent receives a positive reward when the distance is within 10 units, and a negative reward otherwise.

Table 3.2: Reward Function based on Distance

Dist	Reward
≤ 10	1
> 10	-1

In summary, the agent takes observation from the environment and use its neural network to determine the best action to take given the observation. Upon taking the action, the environment sends a positive or negative reward signal to inform the agent of the effect of its action; and the loop continues until the agent learns the best policy.

3.5 Development of the RL Environment

3.5.1 Data Collection for PULSR Environment Specifications

Several episodes of training were conducted on the PULSR robot using the operational modes of operation of the robot - active and passive; the assistive mode which is the third mode is not operational due to the malfunctions in the force sensors. The data collected was every episode is logged automatically into a csv file whose header is as in Figure 3.4

These csv files were collated into different folders according to their modes for further processing.

THRESHOLD	SCORE	UPPER ANGLE	UPPER THRESHOLD	UPPER_LOAD_CELL	LOWER ANGLE	LOWER THRESHOLD	LOWER_LOAD_CELL	DEP	DEM	TIMESTAMP	SAMPLE_NO
1000	1	0	1000	32257	110	1000	32855	249	82	9/8/2025 16:56	0
1000	2	0	1000	32257	110	1000	32809	249	78	9/8/2025 16:56	0
1000	3	0	1000	32258	110	1000	32816	249	75	9/8/2025 16:56	0
1000	4	0	1000	32258	110	1000	32818	249	73	9/8/2025 16:56	0
1000	5	0	1000	32258	110	1000	32813	249	72	9/8/2025 16:56	0
1000	6	0	1000	32256	110	1000	32801	249	71	9/8/2025 16:56	0
1000	7	0	1000	32259	110	1000	32799	249	71	9/8/2025 16:56	0
1000	8	0	1000	32258	110	1000	32802	249	72	9/8/2025 16:56	0
1000	9	0	1000	32259	110	1000	32815	249	73	9/8/2025 16:56	0
1000	10	0	1000	32256	110	1000	32827	249	75	9/8/2025 16:56	0
1000	10	0	1000	32258	110	1000	32821	249	78	9/8/2025 16:56	0
1000	10	1	1000	31990	110	1000	32493	249	82	9/8/2025 16:56	0

Figure 3.4: Header of the Episode Data

3.5.2 Data Processing Pipeline

Taking it from the data collection step, the collected data needed to be processed to extract meaningful information and also feed into the reinforcement learning algorithm. Thus, the data processing pipeline was created. This pipeline is structured into three units: input, output and scripts. The input unit of the pipeline implies the logged episode data separated in folders by mode, while the outputs of the pipeline are three:

- Coordinates of x - y trajectory of the episodes
- Reward of each episode
- Concatenated x - y coordinates

The scripts used to process the input returning the output, and their functions are:

1. **episodes_data_analysis:** From each of the folders containing the CSV data of the episodes of each of the modes, extract the CSV files and convert them to dataframes (DFs) with the same name. Drop irrelevant columns, leaving the *score*, *upper angle*, *lower angle*, *upper load cell*, and *lower load cell*. Compute x and y from *lower angle*, *upper angle*, and the lengths of the links. Apply this process on all CSV data files across all modes. Compute the reward for each episode in a dataframe. Drop all columns leaving only x and y . Store x and y of every episode in separate CSV data files. Store the `reward_df` in a separate CSV data file.
2. **visualize_end_effector_coordinates:** Load all CSV files from the `end_effector_coords` folder into a dictionary of dataframes. Load `reward_df` into a dataframe. Plot all

trajectories with their corresponding rewards in the plot titles, and save all plots in a single PDF file.

3.5.3 Environment Specifications and Assumptions

1. **Description:** A simple 2D path tracking environment where an agent (black box) follows the red ball through the boundaries of the circle in a complete revolution. The goal is to find the shortest trajectory of a complete revolution while maintaining close proximity to the red ball.
2. **Physical Outlook:** As shown in Figure 3.5.
3. **State Space:** x coordinate, y coordinate, and $score$ {score $+= 1$ if $d \leq 70$, where d is the Euclidean distance between the agent and the ball}.
4. **Observation Space:** Same as State Space.
5. **Action Space:** Up, Right, Down, Left ("N", "E", "S", "W"). If the agent hits the workspace boundary, it will reverse the initial step and find a new path.
6. **Reward Function:** As described earlier.

Note: The most significant engineering consideration in defining the specifications of this environment is the kinematics of the robot without considering the dynamics due to the force sensor constraints.

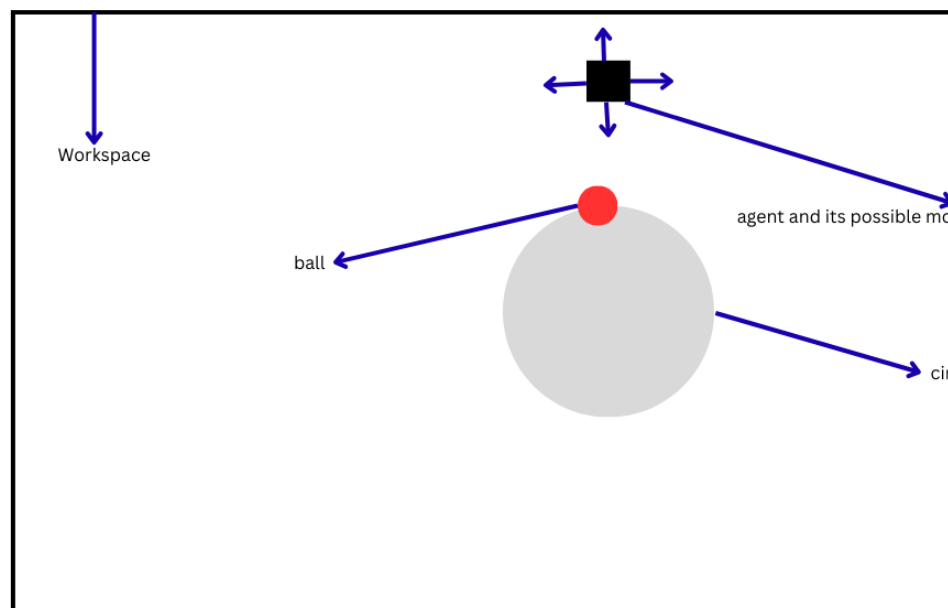


Figure 3.5: Physical Outlook of the Environment

3.5.4 The Standard Gymnasium Environment

Methods of the Environment

Table 3.3: Environment Methods and Their Purpose

Method	Argument(s)	Return Value(s)	Purpose
init	workspace_csv, trajectory_csv	–	Define workspace and trajectory
get_obs	–	[ax, ay, tx, ty, dist]	Get current observation
reset	–	Observation and info	Reset the environment
step	action	Observation, reward, terminated, truncated, info	Execute an action and get the next observation, reward, and other information

Environment Testing

The environment was tested against the standards of gymnasium using the check env function as shown in Figure 3.6. All the four actions were sent repeatedly to the environment and the observations were observed and confirmed to conform with the actual PULSR motions, as shown in Figure 3.7


```

import gymnasium as gym
from gymnasium.envs.registration import register
import pulsrEnv # must import the file that defines PulsrEnv

# Clear old registrations (useful in notebooks)
if "pulsr-virtualenv-v0" in gym.envs.registration.registry:
    del gym.envs.registration.registry["pulsr-virtualenv-v0"]

register(
    id="pulsr-virtualenv-v0",
    entry_point="pulsrEnv:PulsrEnv", # module_name:class_name
)

env = gym.make("pulsr-virtualenv-v0")
print("Env created:", env)

Env created: <OrderEnforcing<PassiveEnvChecker<PulsrEnv<pulsr-virtualenv-v0>>>>

from gymnasium.utils.env_checker import check_env

# Create an instance of the registered environment
env = gym.make("pulsr-virtualenv-v0")

# This will catch many common issues
try:
    check_env(env.unwrapped)
    # check_env(env)
    print("Environment passes all checks!")
except Exception as e:
    print(f"Environment has issues: {e}")
    # print(e)

Environment passes all checks!

```

Figure 3.6: Environment Test with Gymnasium Standards

```
# Test specific action sequences to verify behavior
env = gym.make("pulsr-virtualenv-v0", workspace_csv="workspace_coordinates/workspace_1.csv", trajectory_csv="track_trajectory_coordinates.csv")
obs, info = env.reset(seed=42) # Use seed for reproducible testing

obs_agent = obs[0], obs[1]
obs_target = obs[2], obs[3]

print(f"Starting position - Agent: {obs_agent}, Target: {obs_target}")

# Test each action type
actions = [0, 1, 2, 3] # right, up, left, down
for action in actions:
    old_pos = list(obs_agent).copy()
    obs, reward, terminated, truncated, info = env.step(action)
    new_pos = obs[0], obs[1]
    print(f"Action {action}: {old_pos} -> {new_pos}, reward={reward}")

Starting position - Agent: (np.float32(1.7400862), np.float32(44.8354)), Target: (np.float32(10.267074), np.float32(43.22586))
Action 0: [np.float32(1.7400862), np.float32(44.8354)] -> (np.float32(1.7400862), np.float32(45.8354)), reward=8.917354221797131
Action 1: [np.float32(1.7400862), np.float32(44.8354)] -> (np.float32(2.740086), np.float32(45.8354)), reward=7.966506878764213
Action 2: [np.float32(1.7400862), np.float32(44.8354)] -> (np.float32(2.740086), np.float32(44.8354)), reward=7.697152289943553
Action 3: [np.float32(1.7400862), np.float32(44.8354)] -> (np.float32(1.7400862), np.float32(44.8354)), reward=8.677564626217938

## The actions are confirmed to follow the mapping rule:
### 0 --> North (increase y)
### 1 --> East (increase x)
### 2 --> South (decrease y)
### 3 --> West (decrease x)
```

Figure 3.7: Environment Test with Actions

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 RL Model Training

After developing the environment, there is need to train the agent to learn the best of trasversing the environment mimicing the real PULSR robot. The agent was therefore trained in the environment with a RL model with the following details:

- **Benchmark Model:** Deep Q-Network (DQN)
- **Benchmark Average Reward Per Episode:** 230.5
- **Training Timesteps:** 10000 and 100000
- **Policy Class:** DQNPoly
- **Q-Network (q_net) Artifacts:**
 - Input Layer: 5 features corresponding to the observation vector $[ax, ay, tx, ty, dist]$
 - Two Hidden Layers: 64 neurons each with ReLU activations
 - Output Layer: 4 values corresponding to the 4 possible actions in the environment
- **Target Q-Network (q_net.target):** Same structure as the online network (used for stable training). By keeping the target network stale for a while, DQN avoids instability and oscillation during training.

These are the rollout metrics of the last episode of the 100000 timestep training

- **Rollout:**

- `ep_len_mean`: 238
- `ep_rew_mean`: 232
- `exploration_rate`: 0.05

- **Time:**

- `episodes`: 420
- `fps`: 424
- `time_elapsed`: 235
- `total_timesteps`: 99,960

- **Train:**

- `learning_rate`: 0.0001
- `loss`: 0.0141
- `n_updates`: 24,964

An episode length mean = 238 means that on average, the agent interacts with the environment for 238 timesteps before the episode ends (either by termination or reaching max steps). episode reward mean = 232, this high reward suggests that the agent has learned a fairly effective strategy for following the target (red ball / trajectory). Exploration rate = 0.05: This is the probability of taking a random action (instead of a greedy one). At this point in training, the agent still explores 5 percent of the time to avoid local optima, but mostly exploits the learned Q-values.

4.2 Testing the RL Model

After training the model with different parameters, the model was loaded and tested. For the 10000 timestep model, the test results were woeful as shown in Figure 4.4, Figure 4.5 and Figure 4.6. The 100000 timestep model was tested the same way and the results were promising as shown in the images Figure 4.1, Figure 4.2, Figure 4.3. Since the reward for each action taken is either plus or minus 1, the complete flat plot implies that the end effector followed the target closely from the beginning to the end of the training. The drops in the other plots imply that the end effector went out of proximity in those steps.



Figure 4.1: Excellent Test Result 1

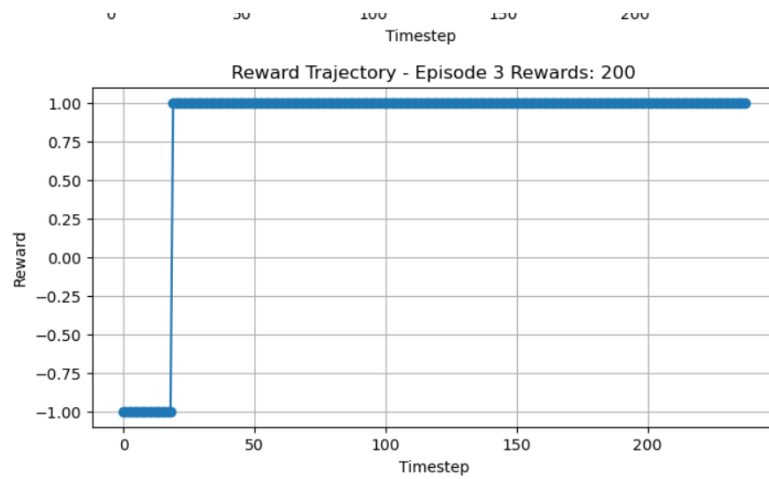


Figure 4.2: Excellent Test Result 2

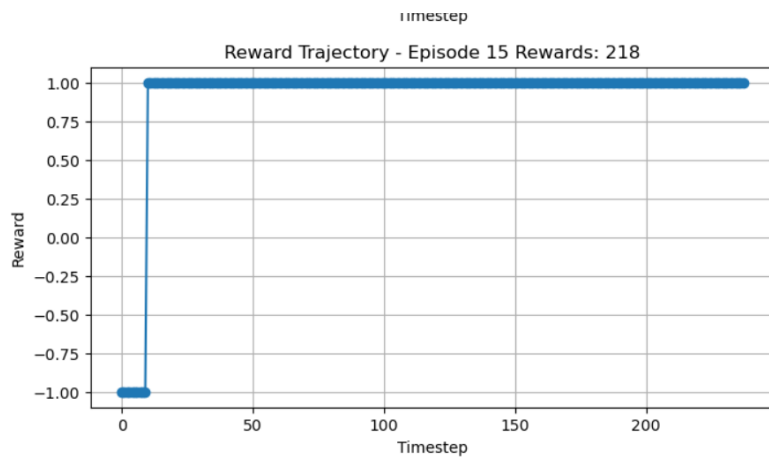


Figure 4.3: Excellent Test Result 3



Figure 4.4: Woeful Test Result 1

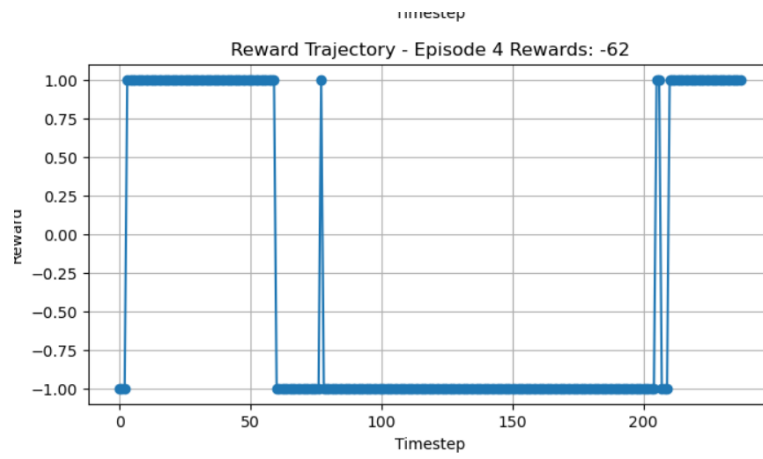


Figure 4.5: Woeful Test Result 2

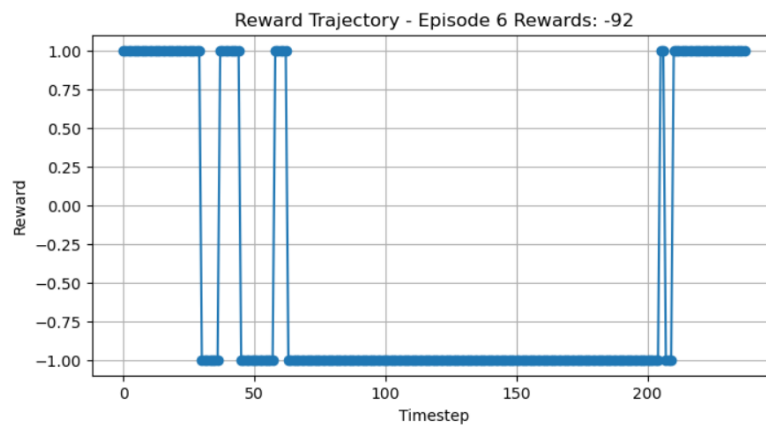


Figure 4.6: Woeful Test Result 3

CHAPTER FIVE

CONCLUSION

5.1 Summary of Work

This project successfully developed a reinforcement learning (RL) control strategy for the Platform for Upper Limb Stroke Rehabilitation (PULSR), a brushless DC motor-actuated parallelogram arm robot. The study began with an extensive review of the state-of-the-art in reinforcement learning for rehabilitation robotics, highlighting the progress and gaps in adaptive and patient-specific control strategies.

The kinematics and dynamics of the PULSR platform were examined to provide a theoretical foundation for environment modeling. Due to the limitations of the force sensors, the environment was modeled using the kinematics alone. An RL problem was formulated with a defined state space, action space, and reward function. A custom RL environment was then developed following Gymnasium standards, ensuring compatibility with modern RL frameworks.

Training was conducted using a Deep Q-Network (DQN) benchmark model. After 100,000 timesteps of training, the model achieved an average episode reward of 230.5, demonstrating its ability to learn effective motion strategies for trajectory tracking. The results showed that the trained agent could successfully follow target trajectories

within the environment, validating the feasibility of reinforcement learning control for the PULSR robot.

5.2 Key Contributions

- **Integration of RL into Rehabilitation Robotics** – Demonstrated the feasibility of reinforcement learning as an adaptive controller for a brushless DC motor-actuated parallelogram arm robot.
- **Development of a Custom RL Environment** – Designed and validated an environment that captures the essential kinematics of PULSR, enabling the training of reinforcement learning algorithms.
- **Performance Evaluation** – Showed that a DQN-based controller can achieve high average rewards and effective trajectory tracking within the modeled environment.

5.3 Challenges Encountered

- **Force Sensor Limitations** – Significant noise and unpredictable non-responsiveness in the force sensors made them unreliable for modeling robot dynamics.
- **Interfacing Issues** – The control application of PULSR is hardcoded in C#, while the reinforcement learning implementation is in Python. This language barrier created significant difficulties in real-time deployment and testing.

5.4 Recommendations for Future Work

- **Incorporation of Dynamics** – Future work should integrate both the kinematics and dynamics of the robot into the RL environment to achieve more realistic modeling and improved control accuracy.
- **Codebase Standardization** – Converting the C# control codebase into Python, or alternatively wrapping it into a fetchable API, would significantly ease the integration of RL algorithms.
- **Exploration of Alternative Frameworks** – Future work could leverage reinforcement learning frameworks for C#, such as TorchSharp, to bridge the language gap without completely rewriting the codebase.
- **Sensor Improvement** – Upgrading the force sensors or applying advanced filtering techniques may resolve noise and responsiveness issues, thereby enabling richer environment modeling and patient-interactive control.

5.5 Conclusion

In conclusion, this project has demonstrated that reinforcement learning can be successfully applied to the control of the PULSR rehabilitation robot. Although limited by sensor constraints and interfacing challenges, the work lays a solid foundation for future studies aimed at achieving adaptive, patient-specific rehabilitation control. With further refinement and integration, reinforcement learning-based controllers hold strong

potential to improve recovery outcomes and enhance the effectiveness of rehabilitation robotics.

REFERENCES

- Ayodele, K., Omolayo, I., Komolafe, E., Jubril, A., Obreba, P., Ogunmodede, A., ... Komolafe, M. (2023, 07). A technique for end-effector force estimation in parallelogram arm robot using link-integrated load cells. *FUOYE Journal of Engineering and Technology*, 8. doi: 10.46792/fuoyejet.v8i2.1035
- Buss, S. (2004, 05). Introduction to inverse kinematics with jacobian transpose, pseudoinverse and damped least squares methods. *IEEE Transactions in Robotics and Automation*, 17.
- Faremi, B., Ayodele, K., Jubril, A., Fakunle, A., Olaogun, M., Fawale, M., & Komolafe, M. (2022, 07). *A simulation study of functional electrical stimulation for an upper limb rehabilitation robot using iterative learning control (ilc) and linear models*. doi: 10.48550/arXiv.2207.07289
- Hayat, A., Chittawadigi, R., Udai, A., & Saha, S. (2013, 07). Identification of denavit-hartenberg parameters of an industrial robot. In (p. 1-6). doi: 10.1145/2506095.2506121
- He, J., Gao, F., & Sun, Q. (2018, 11). Design and kinematic analysis of a novel hybrid kinematic mechanism with seven-degrees-of-freedom and variable topology for operation in space. *Journal of Mechanisms and Robotics*, 11(1), 011003. Retrieved from <https://doi.org/10.1115/1.4041584> doi: 10.1115/1.4041584
- Hong, J., Chun, C., Kim, S.-J., & Park, F. (2019, 04). Gaussian process trajectory learning and synthesis of individualized gait motions. *IEEE Transactions on*

- Neural Systems and Rehabilitation Engineering, PP*, 1-1. doi: 10.1109/TNSRE.2019.2914095
- Kucuk, S., & Gungor, B. (2016a, 10). Inverse kinematics solution of a new hybrid robot manipulator proposed for medical purposes. In (p. 1-4). doi: 10.1109/TIPTEKNO.2016.7863076
- Kucuk, S., & Gungor, B. (2016b, 10). Inverse kinematics solution of a new hybrid robot manipulator proposed for medical purposes. In (p. 1-4). doi: 10.1109/TIPTEKNO.2016.7863076
- Liégeois, A. (1977). Automatic supervisory control of the configuration and behavior of multi-body mechanisms.. Retrieved from <https://api.semanticscholar.org/CorpusID:57402576>
- Merat, F. (1987, 05). Introduction to robotics: Mechanics and control. *Robotics and Automation, IEEE Journal of*, 3, 166 - 166. doi: 10.1109/JRA.1987.1087086
- Shah, F., Khan, N., Jamwal, P., Chetty, G., Goecke, R., & Hussain, S. (2025, 04). Inverse kinematics solution for a six-degree-of-freedom upper limb rehabilitation robot using deep learning models. *Neural Computing and Applications*, 37, 12991-13009. doi: 10.1007/s00521-025-11222-5
- Tolani, D., Goswami, A., & Badler, N. (2000, 10). Real-time inverse kinematics techniques for anthropomorphic limbs. *Graphical models*, 62, 353-88. doi: 10.1006/gmod.2000.0528
- Unzueta, L., Peinado, M., Boulic, R., & Suescun, A. (2008, 09). Full-body performance animation with sequential inverse kinematics. *Graphical Models*, 70, 87-104. doi: 10.1016/j.gmod.2008.03.002
- Wang, A., Wei, Y., Han, H., Guan, L., Zhang, X., & Xu, X. (2018, 05). Ocean wave active

compensation analysis of inverse kinematics for hybrid boarding system based on fuzzy algorithm. In (p. 1-6). doi: 10.1109/OCEANSKOB.2018.8559143

Zhang, Y., Li, S., Nolan, K., & Zanutto, D. (2020, 07). *Reinforcement learning assist-as-needed control for robot assisted gait training.*